

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm

import warnings
warnings.filterwarnings(action='ignore')
```

Extremwerte ohne Nebenbedingungen

Bestimmen Sie alle lokalen Extremwerte (inklusive Sattelpunkte) der folgende Funktionen.

a) $f(x, y) = x^2 - 3xy + xy^3 + 1$

b) $f(x, y) = (x^2 + y^2)e^{-x}$

Lösung

Vorgehensweise:

- Bestimmung der partiellen Ableitungen
- Bestimmung möglicher Extremwerte durch lösen von $f_x = 0$ und $f_y = 0$ (*notwendige Bedingung*)
- Überprüfung der Diskriminante $f_{xx}f_{yy} - f_{xy}^2 > 0$ (*hinreichende Bedingung*)
- Basierend auf f_{xx} entscheiden, ob es ein Minimum ($f_{xx} > 0$) oder Maximum ($f_{xx} < 0$) ist.

a)

- $f_x = 2x - 3y + y^3$, $f_y = -3x + 3xy^2 = 3x(y^2 - 1)$
- mögliche Extremwerte: $\{(-1, -1), (1, 1), (0, 0), (0, \sqrt{3}), (0, -\sqrt{3})\}$
- partielle Ableitungen 2. Ordnung: $f_{xx} = 2$, $f_{xy} = -3 + 3y^2$, $f_{yy} = 6xy$
- Minima: $(-1, -1), (1, 1)$

b)

- $f_x = (2x - x^2 - y^2)e^{-x}$, $f_y = 2ye^{-x}$
- mögliche Extremwerte: $\{(0, 0), (2, 0)\}$
- partielle Ableitungen 2. Ordnung: $f_{xx} = (2 - 4x + x^2 + y^2)e^{-x}$, $f_{xy} = -2ye^{-x}$, $f_{yy} = 2e^{-x}$
- Minimum: $(0, 0)$

```
In [2]: xx,yy = np.meshgrid(np.linspace(-2,2,41), np.linspace(-2,2,61))
f = xx**2 - 3*xx*yy + xx*yy**3 + 1

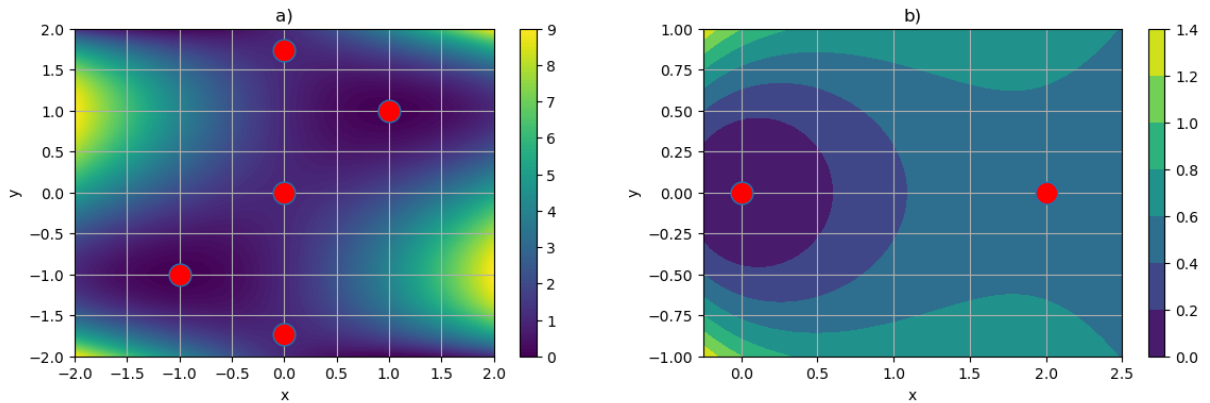
fig = plt.figure(figsize=(14,4))
ax1 = fig.add_subplot(121)
clb = ax1.contourf(xx,yy,f, 101)
ax1.plot([-1,1,0,0,0], [-1,1,0,np.sqrt(3), -np.sqrt(3)], marker='o', linestyle='',\
        markersize=15, markerfacecolor='red')
ax1.set_xlabel('x')
ax1.set_ylabel('y')
ax1.set_title('a')
ax1.grid()
fig.colorbar(clb, ax=ax1)

xx,yy = np.meshgrid(np.linspace(-0.25,2.5,61), np.linspace(-1,1,41))
f = (xx**2 + yy**2)*np.exp(-xx)

ax2 = fig.add_subplot(122)
clb = ax2.contourf(xx,yy,f)
ax2.plot([0,2], [0,0], marker='o', linestyle='', markersize=15, markerfacecolor='red')
ax2.set_xlabel('x')
ax2.set_ylabel('y')
ax2.set_title('b')
ax2.grid()
```

```
fig.colorbar(c1b, ax=ax2)

plt.show()
```



Extremwerte mit Nebenbedingungen

a) Wie lauten die globalen Extremwerte der Funktion $z = x + y$ unter der Nebenbedingung $x^2 + y^2 = 1$?

b) In eine Ellipse mit den Halbachsen a und b soll ein achsenparalleles Rechteck eingefügt werden. Bestimmen Sie die Eckpunkte des Rechtecks so, dass die Fläche des Rechtecks maximal wird.

Hinweis: Eine Ellipse wird durch die Gleichung $(bx)^2 + (ay)^2 - (ab)^2 = 0$ bzw. mittels $\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$ beschrieben

c) Welcher Punkt der Ebene $2x + 3y + z = 14$ hat vom Koordinatenursprung den kleinsten Abstand?

Lösung

Vorgehensweise:

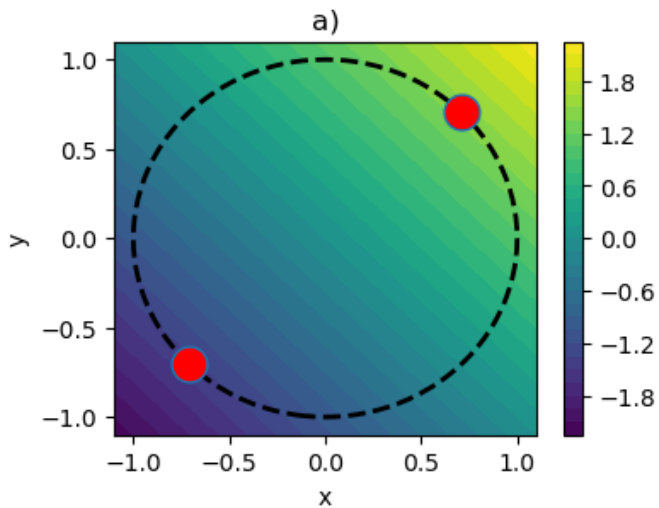
- Aufstellen der Zielfunktion erweitert um Lagrange Multiplikator(en)
- analoge Schritte wie bei Berechnung eines Extremwertes ohne Nebenbedingung

a)

- erweiterte Zielfunktion: $\tilde{z} = x + y + \lambda(x^2 + y^2 - 1)$
- partielle Ableitungen: $\tilde{z}_x = 1 + 2\lambda x$, $\tilde{z}_y = 1 + 2\lambda y$, $\tilde{z}_\lambda = x^2 + y^2 - 1$
- Maximum: $\left(\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2}\right)$
- Minimum: $\left(-\frac{\sqrt{2}}{2}, -\frac{\sqrt{2}}{2}\right)$

```
In [3]: xx,yy = np.meshgrid(np.linspace(-1.1,1.1,41), np.linspace(-1.1,1.1,41))
f = xx + yy

fig = plt.figure(figsize=(4,3))
ax = fig.add_subplot(111)
c1b = ax.contourf(xx,yy,f,32)
ax.contour(xx,yy, xx**2 + yy**2, [1], colors='black', linestyle='--', linewidths=2)
ax.plot([-np.sqrt(2)/2,np.sqrt(2)/2],[-np.sqrt(2)/2,np.sqrt(2)/2], marker='o', linestyle='', markersize=10, markerfacecolor='red')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title('a')
#ax.axis('equal')
fig.colorbar(c1b, ax=ax)
plt.show()
```



b)

Ansatz: Anstatt des gesamten Rechtecks, kann auch nur ein Viertel aus Symmetriegründen betrachtet werden

- erweiterte Zielfunktion: $f(x, y, \lambda) = xy + \lambda [(bx)^2 + (ay)^2 - (ab)^2]$
- partielle Ableitungen: $f_x = y + 2b^2x\lambda$, $f_y = x + 2a^2y\lambda$, $f_\lambda = (bx)^2 + (ay)^2 - (ab)^2$
- Lösungen: $x = \pm \frac{a}{\sqrt{2}}$, $y = \pm \frac{b}{\sqrt{2}}$, $f_{\text{einviertel}} = \frac{ab}{2}$

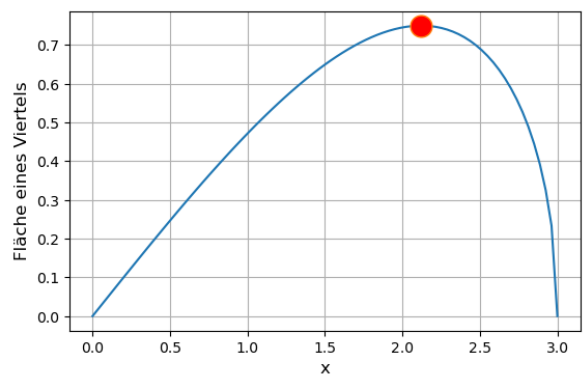
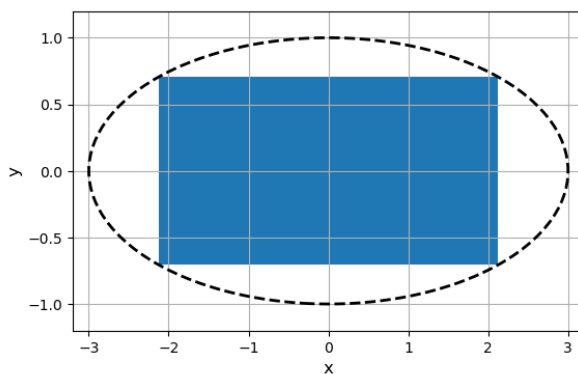
```
In [4]: a = 3
b = 1
x = np.linspace(0,a,81)
y = b*np.sqrt(1-x**2/a**2)

xx,yy = np.meshgrid(np.linspace(-3.2,3.2, 128), np.linspace(-1.2,1.2,48))

fig = plt.figure(figsize=(14,4))
ax1 = fig.add_subplot(121)
ax1.contour(xx,yy,xx**2/a**2 + yy**2/b**2, [1],colors='black', linestyle='--', linewidths=2)
ax1.fill_between([-a/2*np.sqrt(2), a/2*np.sqrt(2)], \
                 [b/2*np.sqrt(2), b/2*np.sqrt(2)],[-b/2*np.sqrt(2), -b/2*np.sqrt(2)])
ax1.set_xlabel('x', fontsize=12)
ax1.set_ylabel('y', fontsize=12)
ax1.grid()

ax2 = fig.add_subplot(122)
ax2.plot(x,x*y/2)
ax2.plot([a/2*np.sqrt(2)], [1/2*a/2*np.sqrt(2)*b/2*np.sqrt(2)], marker='o', markersize=15, \
         markerfacecolor='red')
ax2.set_xlabel('x', fontsize=12)
ax2.set_ylabel('Fläche eines Viertels', fontsize=12)
ax2.grid()

plt.show()
```



c)

- erweiterte Zielfunktion: $f(x, y, z, \lambda) = x^2 + y^2 + z^2 + \lambda(2x + 3y + z - 14)$
- partielle Ableitungen: $f_x = 2x + 2\lambda$, $f_y = 2y + 3\lambda$, $f_z = 2z + \lambda$, $f_\lambda = 2x + 3y + z - 14$
- Darstellung als lineares System:
$$\begin{bmatrix} 2 & 0 & 0 & 2 \\ 0 & 2 & 0 & 3 \\ 0 & 0 & 2 & 1 \\ 2 & 3 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ \lambda \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 14 \end{bmatrix}$$
- Lösung: $x = 2$, $y = 3$, $z = 1$, $\lambda = -2$

```
In [5]: M = np.array([[2,0,0,2], [0,2,0,3], [0,0,2,1], [2,3,1,0]])
b = np.array([[0], [0], [0], [14]])
x = np.linalg.solve(M,b)
print(x)
```

```
[[ 2.]
 [ 3.]
 [ 1.]
 [-2.]]
```

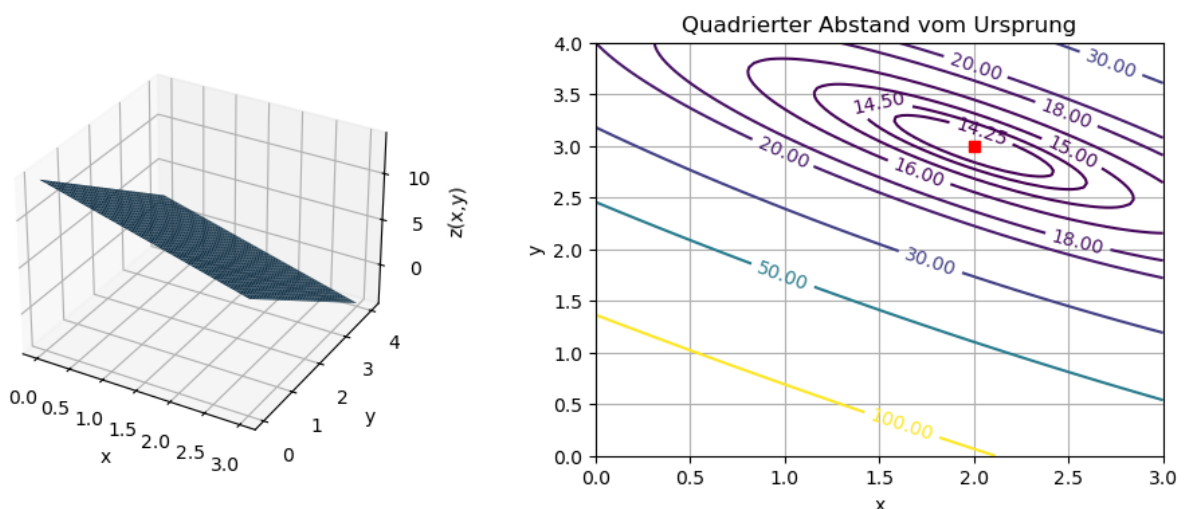
```
In [6]: x = np.linspace(0,3,101)
y = np.linspace(0,4,101)

X,Y = np.meshgrid(x,y)
Z = -2*X - 3*Y + 14

fig = plt.figure(figsize=(12,4))

ax1 = fig.add_subplot(121, projection='3d')
ax1.plot_surface(X,Y,Z)
ax1.set_xlabel('x')
ax1.set_ylabel('y')
ax1.set_zlabel('z(x,y)')

ax2 = fig.add_subplot(122)
cs = ax2.contour(X,Y, X**2 + Y**2 + Z**2, [14.25,14.5,15,16,18,20,30,50,100])
ax2.plot([2],[3], marker='s', color='red')
ax2.grid()
ax2.set_xlabel('x')
ax2.set_ylabel('y')
ax2.set_title('Quadrierter Abstand vom Ursprung')
ax2.clabel(cs, inline=True, fontsize=10)
plt.show()
```



Anwendung

Gegeben sind folgende Messpunkte (x_i, y_i) : $\{(0, 2.9), (1, 5.1), (2, 7.1), (3, 8.8)\}$

Bestimme die entsprechende Ausgleichsgerade, d.h. jene Gerade bei der die Summe der quadrierten Fehler

am kleinsten ist.

Lösung

Geradengleichung ist: $y = mx + b$, wobei m und b bestimmt werden müssen

Summe der quadrierten Fehler ist: $\Delta = \sum_{i=1}^4 (mx_i + b - y_i)^2$

mit den vorgegebenen Messpunkten ergibt sich

$$\Delta = (b - 2.9)^2 + (m + b - 5.1)^2 + (2m + b - 7.1)^2 + (3m + b - 8.8)^2$$

Δ muss minimal werden, daher werden die entsprechenden partiellen Ableitungen benötigt

$$\begin{aligned} \bullet \frac{\partial \Delta}{\partial b} &= 2(b - 2.9) + 2(m + b - 5.1) + 2(2m + b - 7.1) + 2(3m + b - 8.8) = 12m + 8b - 47.8 \\ \bullet \frac{\partial \Delta}{\partial m} &= 2(m + b - 5.1) + 4(2m + b - 7.1) + 6(3m + b - 8.8) = 28m + 12b - 91.4 \end{aligned}$$

Dies führt zum *linearen Gleichungssystem*

$$12m + 8b = 47.8 \text{ und } 28m + 12b = 91.4 \text{ bzw. } \begin{bmatrix} 12 & 8 \\ 28 & 12 \end{bmatrix} \begin{pmatrix} m \\ b \end{pmatrix} = \begin{pmatrix} 47.8 \\ 91.4 \end{pmatrix}$$

Durch Multiplikation der 1. Zeile mit 3 und der 2. Zeile mit -2 und anschließender Addition ergibt sich

$$\begin{bmatrix} 12 & 8 \\ -20 & 0 \end{bmatrix} \begin{pmatrix} m \\ b \end{pmatrix} = \begin{pmatrix} 47.8 \\ -39.4 \end{pmatrix}$$

bzw. lässt sich ablesen: $-20m = -39.4$ bzw. umstellen auf $m = 1.97$

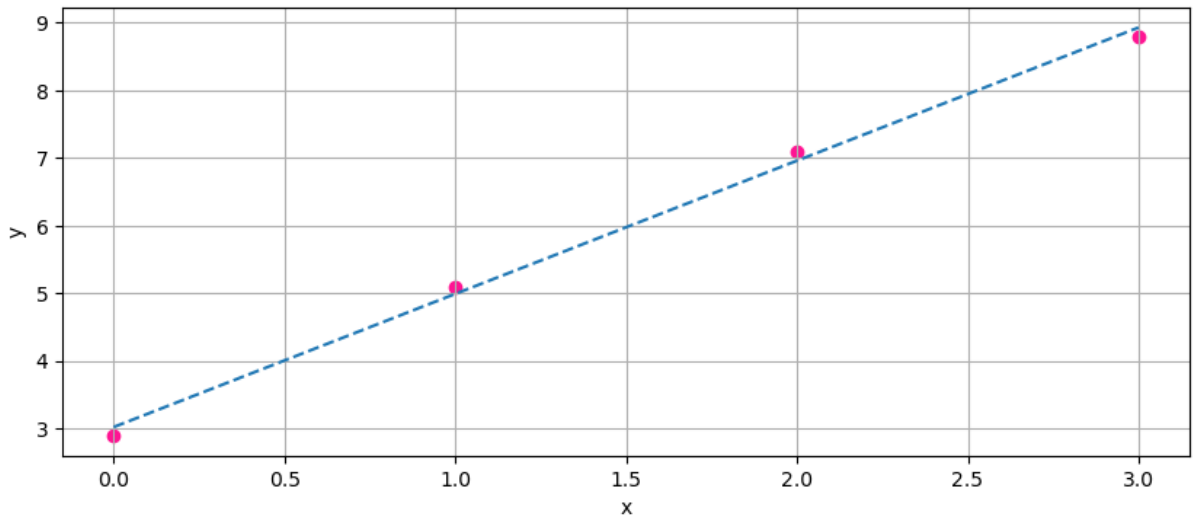
damit ergibt sich aus $12m + 8b = 47.8$ folgendes $b = \frac{47.8 - 12 \cdot 1.97}{8} = 3.02$

Die Ausgleichsgerade lautet schlussendlich: $y = 1.97x + 3.02$

```
In [7]: x = np.array([0,1,2,3])
y = np.array([2.9, 5.1, 7.1, 8.8])

m = 1.97
b = 3.02

fig = plt.figure(figsize=(10,4))
ax = fig.add_subplot(111)
ax.scatter(x,y, color='deeppink')
ax.plot(x, m*x+b, linestyle='--')
ax.grid()
ax.set_xlabel('x')
ax.set_ylabel('y')
plt.show()
```



Optional

Wie können wir ein Optimierungsproblem mit Ungleichungen lösen?

- Wir können entweder die Ungleichung in eine Gleichung umwandeln oder nach extremen Punkten unter der Nebenbedingung der Gleichung suchen und zusätzlich prüfen, welche unbeschränkten lokalen Extrema die Ungleichung erfüllen.

Gibt es eine Möglichkeit, die Ungleichung in eine Gleichung umzuwandeln (zum Beispiel durch Einführung einer zusätzlichen Variable)?

- Ja, wir können die Ungleichung $g(x, y) \leq 0$ in eine Gleichung umwandeln, indem wir eine Variable hinzufügen: $g(x, y) + u^2 = 0$.

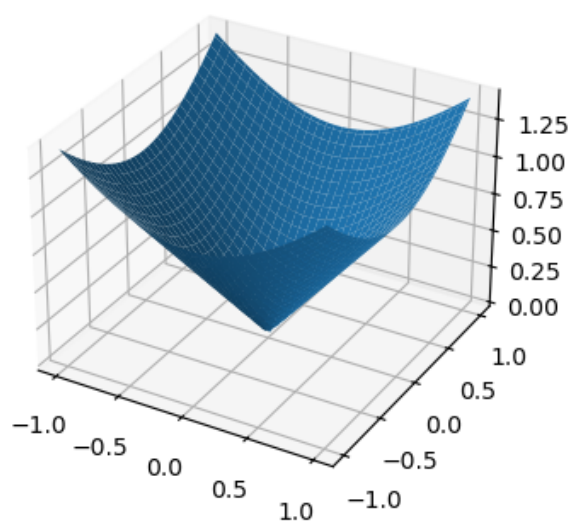
Hat die Funktion $f(x, y) = \sqrt{x^2 + y^2}$ ein Minimum? Können Sie den Gradienten an diesem Punkt berechnen?

- Sowohl x^2 als auch y^2 können nicht negativ sein, sodass der gesamte Ausdruck ebenfalls nicht negativ sein kann und sein Minimum von 0 bei $x = 0$ und $y = 0$ erreicht. Allerdings ist der Gradient an der Stelle $(0, 0)$ nicht definiert: $f_x = \frac{x}{\sqrt{x^2 + y^2}}$, $f_y = \frac{y}{\sqrt{x^2 + y^2}}$.

```
In [23]: x = np.linspace(-1,1,101)
y = np.linspace(-1,1,101)

X,Y = np.meshgrid(x,y)
Z = (X**2+Y**2)**0.5

fig = plt.figure(figsize=(12,4))
ax1 = fig.add_subplot(111, projection='3d')
ax1.plot_surface(X,Y,Z)
plt.show()
```



In []: